

FICHE MÉMO – FILTRES WIRESHARK (TP R102)

Rappel important

- Les filtres Wireshark utilisés ici sont des **filtres d’affichage** (Display Filters)
- Ils servent à **analyser un pcap ou une capture en direct**, pas à capturer

1 Filtres de base indispensables

Filtre	Explication
ip	Affiche uniquement les paquets IP
eth	Affiche uniquement les trames Ethernet
arp	Affiche uniquement le protocole ARP
icmp	Affiche uniquement les messages ICMP (ping)
tcp	Affiche uniquement les segments TCP
udp	Affiche uniquement les datagrammes UDP
dns	Affiche uniquement les requêtes/réponses DNS
http	Affiche uniquement le trafic HTTP

2 Filtres par adresse IP (TP Scapy, Ping, ARP, MITM)

Filtre	Explication
ip.src == 192.31.25.10	Paquets envoyés par cette IP
ip.dst == 192.31.25.1	Paquets destinés à cette IP
ip.addr == 192.31.25.1	Paquets envoyés OU reçus par cette IP

Combinaisons utiles

ip.src == 192.31.25.10 && icmp

Ping envoyé par une machine

ip.dst == 192.31.25.10 && tcp

Connexions TCP reçues par la machine

3 Filtres ARP (TP ARP + ARP Poisoning)

Filtre	Explication
arp	Tout le trafic ARP
arp.opcode == 1	Requêtes ARP (who-has)
arp.opcode == 2	Réponses ARP (is-at)

🔗 ARP Poisoning (TP5)

arp.opcode == 2

👉 Voir les réponses ARP (souvent utilisées pour empoisonner)

arp && eth.src == MAC_ATAQUANT

👉 ARP envoyés par l'attaquant

arp && ip.src == IP_PASSERELLE

👉 ARP prétendant venir de la passerelle (spoof)

4 Filtres ICMP (Ping – TP4)

Filtre	Explication
icmp	Tous les messages ICMP
icmp.type == 8	Echo Request (ping envoyé)
icmp.type == 0	Echo Reply (ping réponse)

🔗 Combinaisons

icmp.type == 8 && ip.dst == 192.31.25.1

👉 Ping envoyé vers la passerelle

icmp.type == 0 && ip.src == 192.31.25.1

👉 Réponse au ping de la passerelle

5 Filtres TCP (Scan de ports – TP4)

Filtre	Explication
tcp	Tout le trafic TCP
tcp.port == 22	Port SSH

<code>tcp.port == 80</code>	Port HTTP
<code>tcp.port == 23</code>	Telnet

Flags TCP (TRÈS IMPORTANT CTP)

Filtre	Signification
<code>tcp.flags.syn == 1</code>	SYN (demande de connexion)
<code>tcp.flags.ack == 1</code>	ACK
<code>tcp.flags.reset == 1</code>	RST (port fermé)

Scan SYN (comme Scapy)

`tcp.flags.syn == 1 && tcp.flags.ack == 0`

 Tentative de connexion (scan)

`tcp.flags.syn == 1 && tcp.flags.ack == 1`

 Port **OUVERT** (SYN-ACK)

`tcp.flags.reset == 1`

 Port **FERMÉ**

Filtres DNS (TP DNS avec Scapy)

Filtre	Explication
<code>dns</code>	Tout le trafic DNS
<code>dns.qry.name</code>	Nom de domaine demandé
<code>dns.flags.response == 1</code>	Réponse DNS

Combinaisons

`dns && ip.dst == 172.18.26.101`

 Requête envoyée au serveur DNS

`dns && dns.flags.response == 1`

 Réponses DNS

`dns.qry.name contains "univ-artois"`

 Requêtes DNS vers ce domaine

7 Filtres HTTP (MITM – TP5)

Filtre	Explication
http	Trafic HTTP
http.request	Requêtes HTTP
http.response	Réponses HTTP
http.request.method == "GET"	Requêtes GET

MITM (ARP Poisoning)

http && ip.src == IP_VICTIME

👉 Navigation web de la victime

http.request

👉 Pages demandées par la victime

http && !(ip.addr == IP_ATAQUANT)

👉 Trafic intercepté (hors attaquant)

8 Combinaisons avancées (niveau CTP)

arp.opcode == 2 && eth.src == MAC_ATAQUANT

👉 Preuve d'ARP poisoning

ip.addr == IP_VICTIME && (http || dns)

👉 Activité web complète de la victime

tcp.flags.syn == 1 && ip.dst == 192.31.25.1

👉 Scan de ports vers la passerelle

icmp || arp

👉 Diagnostic réseau (ping + ARP)

